

CoBrain - Hosting Guide

How to host this platform, which providers fit each piece, and the order they have to come up in. Written against this repository as it stands: `docker-compose.prod.yml`, the Python worker set, and the service URLs the Next.js app actually reads at runtime.

Domain: `cobrain.co`

1. What you are actually hosting

CoBrain is not one app. It is a web tier and a worker tier that share a queue and a database, plus four data services. Most hosting mistakes come from treating it as a single Next.js app and discovering the second half later.

The two halves

Web tier (Next.js)

UI, API routes, OAuth callbacks, RAG orchestration. Stateless - it holds nothing that cannot be rebuilt.

Worker tier (Python)

ingest (consumes the queue, embeds into Qdrant), worker + beat (Celery), search-api (vector retrieval), forecast-api (TFT).

The worker tier is not optional and not an add-on. The web tier only ever pushes onto Redis; if nothing consumes that queue, every upload and every connector sync succeeds, enqueues, and is never embedded. The app looks healthy while learning nothing.

The four data services

PostgreSQL (business)

Workflows, connections, ingested items, brain blocks.

PostgreSQL (auth)

Better Auth's own tables. Deliberately a separate database.

Redis

The ingest queue. Web pushes, Python pops.

Qdrant

Vector store. Written and read by Python only.

Object storage

Uploaded documents and extracted images (S3-compatible).

2. Three ways to host it

A. Single host, everything in Docker (recommended)

One VPS running docker-compose.prod.yml. Caddy terminates TLS on 443; every other service binds to localhost or stays on the compose network. This is what the repository is already built for.

- Cost: roughly \$20-40/month for 4-8 GB RAM (Hetzner, DigitalOcean, Linode).
- Web and workers share a private network - no public hop, no egress cost.
- One deploy target, one secret store, one place to read logs.
- Against it: you own the machine. Backups, patching, and a reboot are yours.
- Scaling is vertical until it isn't - a bigger box, not more boxes.

B. Managed everything, split tiers

Vercel for the web tier, a PaaS for the Python tier, and managed data services. No servers to patch, at the cost of six vendors and public hops between tiers that used to be private calls.

- Cost: realistically \$60-120/month once past free tiers.
- Vercel Hobby cannot run this: crons are once-daily and functions cap at 60s.
- Every tier boundary becomes an authenticated internet call.
- For it: genuinely no ops, and the web tier autoscales.

C. One PaaS for everything

Railway or Render running the web tier, the Python services, and managed Postgres and Redis in one project, on one private network.

- Cost: roughly \$40-80/month.
- Keeps private networking and a single vendor; skips server maintenance.
- The reasonable middle if you do not want to own a VPS.

Recommendation

Start with A. The compose file, the TLS proxy, the health gate and the rollback path already exist and are tested. Move to C if machine maintenance becomes the thing you dislike most. Choose B only if something outside engineering requires Vercel.

3. The order things must come up in

This is a dependency chain, not a checklist you can shuffle. Each step assumes the one above it is already answering.

1. DNS

A records for cobrain.co and www -> the host. Do this first: TLS is issued on the first request, so a certificate cannot be obtained before the name resolves.

2. PostgreSQL (both)

Business and auth databases reachable. Nothing else starts cleanly without them - /api/health returns 503 when Postgres is down, by design.

3. Migrations

prisma migrate deploy runs to completion BEFORE the web tier starts. In compose this is the `migrate` service and the app waits on it. An app started against an older schema fails at the first query, not at boot.

4. Redis

The queue must exist before either tier runs, or enqueues fail and syncs report success having stored nothing.

5. Qdrant + object storage

The collection and the bucket. Workers create what they need on first use, but the endpoints have to answer.

6. Python workers

search-api first (the web tier's RAG calls it), then ingest, worker, beat. Bring ingest up before opening the app to users, or the queue only grows.

7. Web tier

Last. It depends on every one of the above. Point PYTHON_BACKEND_URL at search-api.

8. TLS / reverse proxy

Caddy in front of the web tier. Only this process should publish ports.

9. Scheduler

Celery beat pokes /api/cron/sync and /api/cron/extract every 5 minutes. If beat is not running, use an external pinger with the CRON_SECRET bearer token. Without this, connectors never refresh and uploads are never read.

10. OAuth registration

Register every callback with each provider against the final domain. Do it last, because the URLs depend on the domain being settled.

4. What to use for each piece

If you self-host (A)

Host	Hetzner CX32/CX42, DigitalOcean, or Linode. 4-8 GB RAM.
Everything else	The bundled containers: Postgres, Redis, Qdrant, MinIO.
Auth database	Neon, kept separate as the schema already assumes.
TLS	Caddy - certificates are automatic and renewals need no cron.
Backups	Nightly pg_dump plus a volume snapshot. Untested backups are not backups.

If you split across managed services (B or C)

Web tier	Vercel, Railway, or Render.
Python tier	Railway, Render, or Fly.io. Must be reachable over HTTPS.
PostgreSQL	Neon or Supabase.
Redis	Upstash - the existing ioredis client speaks its rediss:// endpoint.
Qdrant	Qdrant Cloud. Set on the Python side only; the web tier never queries it.
Object storage	Cloudflare R2 or S3. R2 has no egress fees.

The trap in this column

In development every one of these points at localhost. On a split deployment localhost means the container itself, so REDIS_URL, PYTHON_BACKEND_URL and MINIO_ENDPOINT must all be changed. Miss one and the failure is partial: the app loads, sign-in works, and only chat or only uploads are broken.

5. Failure modes worth knowing before they happen

The queue with no consumer

Web tier up, ingest worker down. Uploads return 200, connectors report items synced, and nothing is ever embedded or answerable. Nothing errors. Check the queue depth on `/brain/visualize`, or `LLEN company_brain:ingest`.

Nothing pokes the cron

Connectors only refresh when `/api/cron/sync` is called, and uploads are only read when `/api/cron/extract` is. On Vercel Hobby, crons run once a day, which is not a schedule this app can use. Beat or an external pinger is mandatory.

A localhost OAuth redirect in production

The button works, the request leaves, and the user lands on the provider's error page. Nothing in this app logs it. Redirects are now derived from `APP_URL` and the boot check refuses to start if one still points at localhost or plain http.

Migrations racing the app

Start the web tier and the migration at once and the app serves requests against a schema mid-change. Compose orders this correctly; a manual deploy must too.

The model bill

Ingestion calls Claude per document. Roughly \$0.17 for a 27-page PDF on Opus, about \$0.03 on Haiku. Set a spend limit in the Anthropic console before connecting a large workspace, not after.

6. Rough monthly cost

Excluding model usage, which is per-document and per-question.

A. Single VPS	\$20-40 - one box, all services
C. One PaaS	\$40-80 - managed, private networking, one vendor
B. Fully split	\$60-120 - six vendors, most operational surface

Model usage sits on top of all three and is the line most likely to surprise you. A demo is a few dollars; a large workspace ingested in one pass is not.

Final checklist

- DNS resolves for the apex and www before the first deploy.
- Migrations run to completion before the web tier starts.
- The ingest worker is running - not just search-api.
- Something pokes both /api/cron/* endpoints every few minutes.
- No REDIS_URL, PYTHON_BACKEND_URL or MINIO_ENDPOINT still says localhost.
- Every OAuth callback registered against the production domain.
- A spend limit set in the Anthropic console.
- A backup taken, and restored once to prove it works.